

Class 2

Thursday, September 10

Prediction, Generalization, and the Bias–Variance Trade-Off

Upcoming

Reading (complete by Tuesday, September 15): [ISL textbook](#) Chapter 3

Video (optional) (complete by Tuesday, September 15): [3.1 Simple Linear Regression](#); [3.3 Multiple Linear Regression](#)

Homework (due Thursday, September 17 at 12:59 p.m.): Homework 1

Key takeaways

- Expected test error is a measure of generalization for prediction problems. Test error measures expected performance on new, unseen observations that are ‘like’ (but not the same as!) the training data.
- We usually can’t compute expected test error directly, but we can estimate it with test data that was not used to train the prediction method.
- Under squared loss, $f(x) = \mathbb{E}[Y \mid X = x]$ has the lowest test error (and is therefore ‘best’ in a certain sense). We usually don’t know f , and so try to estimate it with some $\hat{f}$.
- More flexible methods typically have lower bias and higher variance, while less flexible methods typically have higher bias and lower variance. For KNN, smaller values of K are more flexible. Good test-set performance requires balancing bias and variance.

Summary notes

Recap: Supervised Regression Data-Generating Process from Last Lecture. In statistics, we usually start with the data. In supervised learning problems, the training data are

$$\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\}.$$

We then tell a story about how the data came to be. In the first class, I referred to this as a model. But because the term model is over-loaded, moving forward I’ll try to use the phrase ‘data-generating process’, since the story tells us something about how the data could be generated. In lecture 1, we introduced the supervised regression data-generating process:

In the first class and in the textbook, we saw the following data-generating process, that we will continue to use.

$$Y = f(X) + \epsilon,$$

where f is fixed but unknown, X and ϵ are independent, and

$$\mathbb{E}[\epsilon] = 0.$$

This model says that the output can be obtained by taking the input, applying a function to it, then adding some noise.

Independence of X and ϵ implies $\mathbb{E}[\epsilon \mid X = x] = \mathbb{E}[\epsilon] = 0$, so

$$f(x) = \mathbb{E}[Y \mid X = x].$$

We will also often assume that all n observations are independent and identically distributed copies of (X, Y) .

Prediction Goal. We want to use the training data, $\mathcal{D}$, to learn a function $\hat{f}_{\mathcal{D}}$ such that

$$Y \approx \hat{f}_{\mathcal{D}}(X)$$

on *new* observations. We use the subscript to indicate our predictor depends on the training data we observed.

Measuring prediction quality with a loss function. A *loss function* or simply a *loss* $L(y, a)$ assigns a cost to predicting a when the observed response is y . Usually, a loss is large if a and y are very different, and close to 0 if they are similar. The choice of a loss function is in some sense a value judgement about what matters in a prediction problem. In practice (for better or worse), people often use squared-error loss as a ‘default’ in regression problems:

$$L(y, a) = (y - a)^2.$$

For regression problems in this course, we will mostly use squared loss, which is also the focus of the book. Squared loss is mathematically convenient, and at least has the property that it is 0 if and only if our predictions agree with the observed data, and larger otherwise. But other losses might be more appropriate for a particular application, and ideally a loss would reflect how predictions will be used in decision-making or some other downstream application.

What properties might you want from a loss function for regression?

Can you suggest a loss function for a classification problem where there are two possible categories? E.g. if the task is to classify whether an image is a 0 or a 1.

Training error. The *training error* of a prediction method is its average loss on the training data. For squared loss, this is also referred to as the *training mean squared error (training MSE)*, and is equal to:

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}_{\mathcal{D}}(x_i))^2.$$

At first, this is an appealing measure of prediction quality: it is easy to compute, and it is small if the predictions are close to the observed responses. But training error is not a good measure of generalization, which is what we really care about. In fact, later today we will see a prediction method that has training MSE equal to 0 whenever the training inputs are distinct, but often generalizes poorly. This is an example of *overfitting*: fitting the training observations very closely without achieving comparably good performance on new observations. The reason training error is not a good measure of generalization is that it measures how well a prediction method fits data that it has already seen, but we care about how well it will perform on new data that it has not seen.

Test error and generalization. Let (X_0, Y_0) be a new observation drawn from the same population as the training data but independent of $\mathcal{D}$. Once a particular prediction method $\hat{f}_{\mathcal{D}}$ has been learned, its expected loss on this new observation is:

$$R(\hat{f}_{\mathcal{D}} \mid \mathcal{D}) = \mathbb{E}_{X_0, Y_0} [L(Y_0, \hat{f}_{\mathcal{D}}(X_0)) \mid \mathcal{D}].$$

This expectation describes average performance of a fitted prediction method on *unseen data*. Remember that the training data are i.i.d. copies of (X, Y) . Therefore, $R(\hat{f}_{\mathcal{D}} \mid \mathcal{D})$ is *random* because it depends on the random training data. We can also average over the training data to get the *expected test error*:

$$\mathbb{E}_{\mathcal{D}} [R(\hat{f}_{\mathcal{D}} \mid \mathcal{D})] = \mathbb{E}_{\mathcal{D}, X_0, Y_0} [L(Y_0, \hat{f}_{\mathcal{D}}(X_0))].$$

Expected test error is a measure of *generalization*: how well a prediction method learned from the training data performs on new observations that look like the training data, in the sense that they are drawn from

the same population. A method generalizes well when patterns learned from the training sample continue to give low loss on these new observations.

With squared loss, expected test error is referred to as *test mean squared error*.

The textbook uses *test MSE* for the empirical mean squared error computed on a *particular* test set, which we discuss next.

Estimating expected test error with a test set. We usually can't compute the expected test error directly, because we need to average over all possible new observations. But if we have an independent *test set* that was not used to train the model, we can estimate test error by computing the average loss on this test set. In particular, if we have m test observations $(x_1^{\text{test}}, y_1^{\text{test}}), \dots, (x_m^{\text{test}}, y_m^{\text{test}})$, that are all independent of the training data and each other, and have the same distribution as the training data, then under squared loss the test MSE is

$$\frac{1}{m} \sum_{i=1}^m \left(y_i^{\text{test}} - \hat{f}_{\mathcal{D}}(x_i^{\text{test}}) \right)^2.$$

If the test set is used to fit the model (e.g., to choose hyperparameters), then the test MSE is no longer an unbiased estimate of expected test error.

Optimal prediction under squared loss. Let g be any prediction rule. Its squared-error risk for a new predictor-response pair (X, Y) is

$$R(g) = \mathbb{E}[(Y - g(X))^2].$$

Conditioning on X and applying the law of iterated expectations gives

$$\begin{aligned} R(g) &= \mathbb{E}_X[\mathbb{E}[(Y - g(X))^2 \mid X]] \\ &= \mathbb{E}_X[\mathbb{E}[(f(X) - g(X) + \epsilon)^2 \mid X]] \\ &= \mathbb{E}_X[(f(X) - g(X))^2 + \text{Var}(\epsilon \mid X)] \\ &= \mathbb{E}_X[(f(X) - g(X))^2] + \text{Var}(\epsilon). \end{aligned}$$

The cross-product vanishes because $\mathbb{E}[\epsilon \mid X] = 0$. The last equality uses the independence of X and ϵ . The second term does not depend on g , while the first is nonnegative and equals zero when $g(X) = f(X) = \mathbb{E}[Y \mid X]$. Thus the full risk is minimized by $g(x) = f(x) = \mathbb{E}[Y \mid X = x]$ (up to values of x that occur with probability zero). Even this optimal rule cannot predict ϵ ; its remaining error is the *irreducible error*.

Can you remember (or think of) any way to estimate $\mathbb{E}[Y \mid X = x]$ based on a training data set?

Bias–variance Trade-Off. We don't know the true function f , so we can't use this 'optimal' prediction method. It's therefore important to have an understanding of the trade-offs involved in estimating f from data.

Let (X_0, Y_0) be a new test observation, independent of the training set $\mathcal{D}$, with $Y_0 = f(X_0) + \epsilon_0$. At a fixed x , define the average fitted value over possible training sets by

$$\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)].$$

Add and subtract $\bar{f}(x)$ from the prediction error:

$$Y_0 - \hat{f}_{\mathcal{D}}(x) = \epsilon_0 + (f(x) - \bar{f}(x)) + (\bar{f}(x) - \hat{f}_{\mathcal{D}}(x)).$$

Squaring and averaging over both the new response and possible training sets gives

$$\begin{aligned}
& \mathbb{E}_{\mathcal{D}, Y_0} \left[(Y_0 - \hat{f}_{\mathcal{D}}(x))^2 \mid X_0 = x \right] \\
&= \mathbb{E}[\epsilon_0^2 \mid X_0 = x] + (f(x) - \bar{f}(x))^2 + \mathbb{E}_{\mathcal{D}} \left[(\bar{f}(x) - \hat{f}_{\mathcal{D}}(x))^2 \right] \\
&\quad + 2(f(x) - \bar{f}(x))\mathbb{E}[\epsilon_0 \mid X_0 = x] \\
&\quad + 2\mathbb{E}_{\mathcal{D}, Y_0} [\epsilon_0(\bar{f}(x) - \hat{f}_{\mathcal{D}}(x)) \mid X_0 = x] \\
&\quad + 2(f(x) - \bar{f}(x))\mathbb{E}_{\mathcal{D}} [\bar{f}(x) - \hat{f}_{\mathcal{D}}(x)].
\end{aligned}$$

Each cross-product is zero. In particular,

$$\mathbb{E}[\epsilon_0 \mid X_0 = x] = 0, \quad \mathbb{E}_{\mathcal{D}} [\bar{f}(x) - \hat{f}_{\mathcal{D}}(x)] = 0.$$

Moreover, the new test observation is independent of $\mathcal{D}$, so the expectation in the middle cross-product factors into the product of these two zero expectations. Therefore,

$$\begin{aligned}
& \mathbb{E}_{\mathcal{D}, Y_0} \left[(Y_0 - \hat{f}_{\mathcal{D}}(x))^2 \mid X_0 = x \right] \\
&= \underbrace{\text{Var}(\epsilon_0 \mid X_0 = x)}_{\text{irreducible noise}} + \underbrace{(\bar{f}(x) - f(x))^2}_{\text{squared bias at } x} + \underbrace{\text{Var}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}(x))}_{\text{variance at } x}.
\end{aligned}$$

Finally, average over the distribution of X_0 . Because X_0 and ϵ_0 are independent, the irreducible-noise term is constant in x , and the overall expected test MSE is

$$\begin{aligned}
& \mathbb{E}_{\mathcal{D}, X_0, Y_0} \left[(Y_0 - \hat{f}_{\mathcal{D}}(X_0))^2 \right] \\
&= \underbrace{\text{Var}(\epsilon_0)}_{\text{irreducible noise}} + \underbrace{\mathbb{E}_{X_0} \left[(\bar{f}(X_0) - f(X_0))^2 \right]}_{\text{integrated squared bias}} + \underbrace{\mathbb{E}_{X_0} \left[\text{Var}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}(X_0)) \right]}_{\text{integrated variance}}.
\end{aligned}$$

The expected test MSE decomposes into three terms: irreducible noise, integrated squared bias, and integrated variance. The irreducible noise is the same for all prediction methods, so we can only try to minimize the bias and variance terms. The variance term is large when the fitted function $\hat{f}_{\mathcal{D}}$ varies a lot across different training sets, while the bias term is large when the average fitted function $\bar{f}$ is far from the true function f . In general, more flexible methods have lower bias and higher variance, while less flexible methods have higher bias and lower variance. The best prediction method balances these two errors to minimize expected test MSE.

Estimating f with nearest neighbors. We want to estimate $f(X) = \mathbb{E}[Y \mid X]$. If we had lots of input-output pairs (X, Y) with X exactly equal to x , we could estimate $f(x)$ by averaging the corresponding Y values. In practice, we usually don't have many (or any) training inputs exactly equal to x . But we might have some training inputs that are close to x . *K-nearest neighbors* (KNN) regression estimates $f(x)$ by averaging the responses corresponding to these nearby inputs.

For a prediction point x , let $\mathcal{N}_K(x)$ contain the K training inputs closest to x . Define

$$\hat{f}_K(x) = \frac{1}{K} \sum_{i: x_i \in \mathcal{N}_K(x)} y_i.$$

“Closest” requires a notion of distance. Typically, we will use the usual (Euclidean) distance. But when fitting a KNN model, it's important to consider the scale of each predictor, since a predictor with a larger scale will dominate the distance calculation. This scaling is often not obvious, for example one predictor could be measured in dollars and another in years. A common approach is to standardize each predictor to have mean 0 and variance 1 before computing distances.

Increasing K averages over a wider area, which can introduce bias since f may vary over that area. But increasing K also reduces variance, since averaging over more responses reduces the effect of the noise ϵ . Thus small K corresponds to a more flexible KNN fit, while large K corresponds to a less flexible fit.

KNN Regression Simulation. The left panel below compares the extremes $K = 1$ and $K = n$ with the intermediate choice $K = 10$. The right panel repeats the data-generating process 250 times and estimates the integrated squared bias and variance for each K . The panels use separate color palettes because the curves have different interpretations; within each panel, line types and plotting symbols reinforce the color encoding.

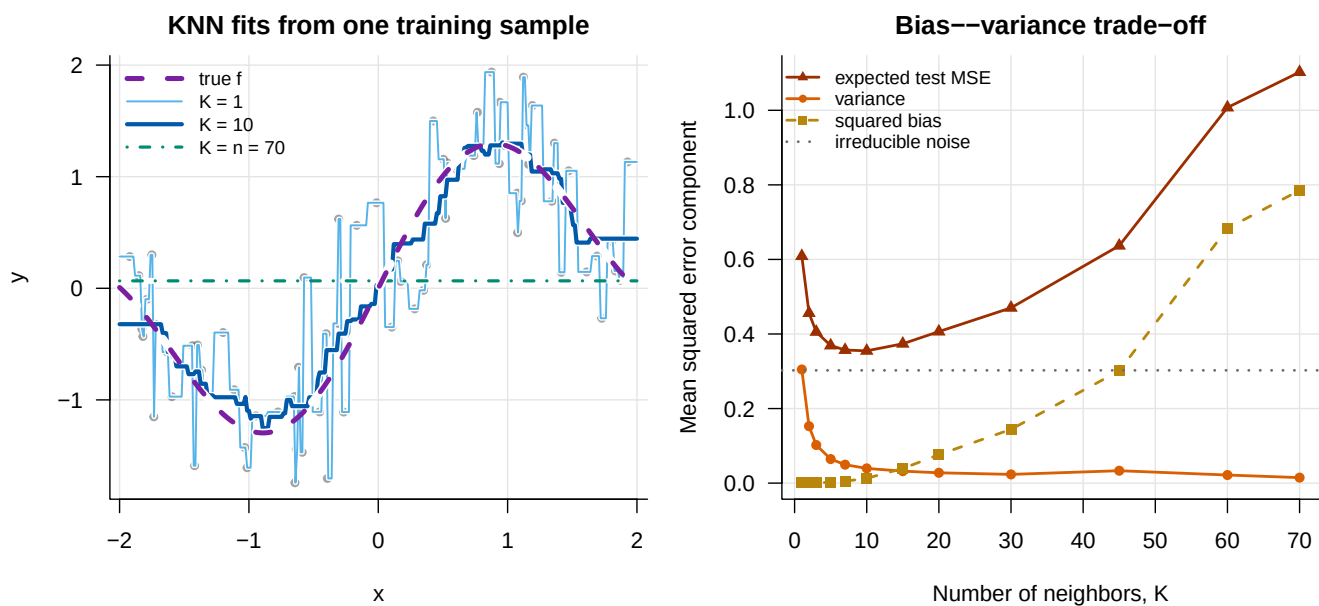


Figure 1: KNN flexibility and the bias–variance trade-off. $K = 1$ follows individual observations closely, $K = n$ is the constant sample mean, and an intermediate K balances the two errors. The simulation uses $n = 70$, $\sigma = 0.55$, and 250 independently generated training sets.

Limitations of KNN.

- *Extrapolation:* KNN just predicts the mean using nearby points. If no points are nearby, there's not really any reason to think its predictions will be good.
- *High-dimensional predictors:* As the number of predictors grows, basically any new point you want to make predictions at will be very different along several dimensions from the observed data.